

航空应急仿真培训平台

开发执行手册（V2.0 · 超详细版）

首期：气囊顶升 Web 培训模块 | 长期：多机型、灭火 VR、力反馈与实体设备

本版与 V1.0 的区别

V1.0 已经确定了 React + Babylon.js + C++20/WASM + ASP.NET Core + PostgreSQL + Blender 的长期架构，以及 16 周首期目标。本版不再讲“学什么”，而是把开发拆到可直接执行的工程步骤：目录、文件、数据结构、接口、模型处理、交互、测试、部署和每日里程碑。

版本日期：2026-09-01

首期执行对象：以 B737-800 + 单主起落架塌陷场景作为可落地示例；正式启动时如更换子型号，仅替换资产/配置包，不改变任务结构。

0. 文档目的与使用方法

这不是“技术选型说明”，而是一份可以直接照着推进的工程执行手册。每个任务都明确：做什么、用什么、具体操作、产物、验收条件、前置依赖和后续关系。

- 首期周期按 16 周、80 个工作日拆解；每 2 周必须形成一个可运行版本。
- 首期只交付 1 个具体机型 + 1 个典型故障的完整闭环，不在四个月内扩到八机型、VR 或真实力反馈。
- 长期架构从第一天保留：Web 端与未来 Unreal VR 端共享 Simulation Core、事件模型、规则版本、后端 API 和资产源文件。
- 任何航空器顶升点、压力阈值、安全区、禁止区和操作规则必须来源于经确认资料；开发阶段可以使用 Draft 数据，但 Draft 数据不得进入正式考核配置。

首期成功判定

到第 16 周，用户能从网页创建任务，进入三维训练，完成气囊铺设、管路连接、分级充气、同步顶升、稳压、泄压与回收；系统能判错、评分、记录事件、回放并在指定终端稳定运行。

1. 固定技术栈与模块边界

模块	固定技术	首期职责
Web 业务平台	React + TypeScript + Vite + React Router + TanStack Query + Zustand + Ant Design	登录、任务、成绩、回放、管理页；只处理 UI 和业务调用。
Web 三维训练	Babylon.js（作为独立 TS package 嵌入 React）	飞机、气囊、管路、点位、动画、摄像机、HUD；不写业务规则。
仿真核心	C++20 + CMake；WASM + Native 双构建	状态机、规则、准物理、评分输入、事件状态；浏览器和未来 Unreal 共用。
后端	ASP.NET Core / C# + OpenAPI	身份、目录、任务、会话、事件、成绩、回放索引、资源清单。
数据库	PostgreSQL + EF Core + JSONB	版本化机型/场景/规则、会话、事件、快照、成绩和审计。
三维资产	Blender + Substance 3D Painter（可选）	源模型清理、拆分、LOD、碰撞代理、材质、GLB、Web 资产包。
资源优化	glTF/GLB + KTX2/BasisU + Meshopt/Draco（按测试选择）	降低网络体积、内存和 GPU 压力。
部署	Docker Compose + Nginx + HTTPS	首期单服务器；Web、API、PostgreSQL、静态资产统一部署。
未来 VR	Unreal Engine 5 + OpenXR + C++	灭火、烟雾、高沉浸 VR、复杂视觉与实体设备。
硬件接口	C++ Adapter Layer	力反馈、追踪器、传感器、实体灭火器、操作台。

2. 最终工程目录与“单一真源”规则

aviation-training/

└─ apps/	
└─ web/	# React 业务网站
└─ api/	# ASP.NET Core API
└─ packages/	
└─ web-sim/	# Babylon.js 三维训练包
└─ contracts/	# JSON Schema / TS 类型 / API DTO 约定
└─ design-tokens/	# 颜色、字号、间距、状态色
└─ core/	
└─ simulation-core/	# C++20; WASM + Native
└─ assets/	
└─ source/	# 原始高质量模型，永远保留
└─ web/	# GLB/KTX2/LOD 发布包
└─ metadata/	# 顶升点、区域、场景、设备、版本
└─ tools/	
└─ blender/	# Blender 批处理脚本
└─ asset-validator/	# Python 资产检查
└─ config-validator/	# JSON Schema 校验
└─ deploy/	
└─ docker-compose.dev.yml	
└─ docker-compose.prod.yml	
└─ nginx/	
└─ tests/	
└─ e2e/	
└─ replay/	
└─ performance/	
└─ docs/	
└─ baseline/	
└─ api/	
└─ database/	
└─ asset-standard/	
└─ acceptance/	

- 规则真源：core/simulation-core；前端不得复制一份“是否正确”的 if/else。
- 接口真源：OpenAPI + packages/contracts；前端禁止手工猜测后端 DTO 字段。
- 三维源资产真源：assets/source；Web 减面模型永远不是唯一源文件。
- 历史训练真源：TrainingEvent + Snapshot + Version；回放不得依赖“当前最新规则”。
- 设计真源：Figma + design-tokens；React 与 Babylon HUD 共享同一套 Token。

3. 首期数据包与文件规范

为了让后续机型扩展变成“换内容包”而不是“复制代码”，首期 B737-800 的全部业务数据必须落成文件和版本。

文件	首期必须包含
aircraft.json	机型、子型号、尺寸、资源版本、审核状态、默认相机参数
lifting-points.json	顶升点 ID、坐标、类型、适用设备、版本、来源
regions.json	安全布设区、禁止区、人员区、区域多边形/包围体
devices.json	气囊、软管、阀门、压力表等训练设备规格
scenario.json	故障类型、初始姿态、可用设备、目标状态、规则版本
rules.json	RuleCode、触发条件、严重度、动作、扣分、版本

文件	首期必须包含
asset-manifest.json	LOD 文件、纹理、哈希、大小、资源 URL、兼容等级

重要

本手册把软件实现步骤写得很细，但不会替你们虚构真实航空器救援参数。顶升点坐标、禁止区、安全阈值、气囊设备性能和操作规则必须通过业务资料或专家审核后才能从 Draft 升级为 Approved。

4. 模块任务执行矩阵

以下每一行都可以直接作为项目管理工具中的独立任务。状态只允许：Todo → Doing → Review → Accepted。只有达到“输出/验收”才算完成。

M0 需求、资料与配置基线

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M0-01 冻结首期对象	docs/baseline/scope.md	明确具体子型号、典型故障、训练模式、运行终端、浏览器、是否要求平板。首期示例固定为 B737-800 + 单主起落架塌陷；若业务资料不足，先换资料最完整子型号，不允许用“B737”大类含糊代替。	scope.md 经确认；后续模型、规则和验收都能引用同一对象。前置：无。后续：M0-02、M6-01、M10-01。
M0-02 建立资料台账	docs/baseline/source-register.xlsx / md	逐项登记飞机资料、顶升点、禁止区、气囊规格、管路图、训练步骤。每项记录来源、日期、版本、许可、审核人和状态 Draft/Approved/Rejected。	所有进入配置的数据都可追溯。前置：M0-01。后续：M0-03、M6-02、M3-04。
M0-03 统一术语和 ID	packages/contracts/naming.md	定义 AircraftVersionId、ScenarioVersionId、RuleSetVersionId、AssetVersionId、DeviceId、RuleCode、ObjectId 等格式。禁止页面显示名直接作为数据库主键。	所有模块 ID 规则一致。前置：M0-01。后续：M5、M4、M3、M2。
M0-04 拆解标准训练流程	docs/baseline/training-flow.md	把流程拆为 Prepare→BagSelect→BagPlace→HoseConnect→LeakCheck→Inflate→Lift→Hold→Deflate→Recover→Complete。每步写前置条件、允许操作、完成条件、阻断条件。	流程表可直接转成状态机配置。前置：M0-02。后续：M3-02、M7-01。
M0-05 建立规则清单	assets/metadata/.../rules.draft.json	把每个错误写成独立 RuleCode，例如 BAG_OUTSIDE_ALLOWED_REGION、HOSE_WRONG_PORT、VALVE_OPEN_BEFORE_CONNECT、PRESSURE_OVER_LIMIT、SYNC_DEVIATION_HIGH。先填结构，不确认的阈值标 Draft。	规则覆盖正常/警告/阻断/失败四级。前置：M0-04。后续：M3-03、M7-01。
M0-06 建立设备清单	assets/metadata/.../devices.json	为每种气囊、软管、阀门、压力表建立 DeviceId、模型资源、接口类型、允许连接对象、教学参数和版本。	三维资产和规则能使用同一 DeviceId。前置：M0-02。后续：M6-03、M2-05、M3-04。
M0-07 建立验收矩阵	docs/acceptance/acceptance-matrix.xlsx / md	把功能改写成可执行用例：输入、操作、预期状态、应触发规则、应记录事件、预期得分、回放结果。至少包含正常流程和 10 类错误流程。	每个首期功能都有验收用例。前置：M0-04~06。后续：M10 全部。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M0-08 确定性能预算	docs/acceptance/performance-budget.md	设定首版预算：Web 壳、单机型资产包、纹理分辨率、三角面、Draw Call、首载时间、PC/平板帧率和内存阈值。数值作为工程预算，可在第 12 周按目标设备实测调整。	M6 和 M2 有明确优化目标。前置：M0-01。后续：M6-04、M2-08、M10-04。

M8 工程仓库、开发环境与 CI/CD

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M8-01 创建 monorepo	Git + pnpm workspace + .NET solution + CMake	初始化仓库，创建 apps/packages/core/assets/tools/deploy/tests/docs 目录；main 只放可交付版本，dev 为集成分支，feature/* 为任务分支。	空仓库结构可 clone 后直接识别。前置：无。后续：所有开发。
M8-02 统一格式与提交规则	.editorconfig、Prettier、ESLint、clang-format、dotnet format	加入 UTF-8、换行、缩进、命名、提交说明模板；CI 中执行格式检查，避免多人环境产生无意义 diff。	四类代码有统一格式。前置：M8-01。后续：CI。
M8-03 搭建本地 PostgreSQL	Docker Compose	创建 docker-compose.dev.yml，启动 PostgreSQL；数据库名 aviation_training_dev；密码通过 .env.local 注入，不写进仓库。	docker compose up 后数据库健康。前置：M8-01。后续：M5-01、M4-01。
M8-04 创建 Web 工程	Vite + React + TypeScript + pnpm	创建 apps/web；安装 Router、TanStack Query、Zustand、Ant Design；建立 src/app、pages、components、services、features、styles。	pnpm dev 能打开空首页。前置：M8-01。后续：M1-01。
M8-05 创建 API 工程	.NET / ASP.NET Core	创建 apps/api；启用 OpenAPI、HealthChecks、ProblemDetails、分环境 appsettings；加入 PostgreSQL EF Core Provider。	dotnet run 后 /health 与 OpenAPI 可访问。前置：M8-01、M8-03。后续：M4-01、M5-01。
M8-06 创建 Simulation Core 工程	C++20 + CMake + GoogleTest/Catch2 + Emscripten	建立 core/simulation-core/src/include/tests/adapters；先编译 Native，再建立 WASM target；核心代码与浏览器绑定代码分目录。	Native 与 WASM 均输出 Hello World。前置：M8-01。后续：M3-01。
M8-07 创建 web-sim package	Babylon.js + TypeScript	packages/web-sim 作为可被 apps/web 引用的包；安装 Babylon core/loaders/gui；导出 TrainingCanvas、SceneController、AssetLoader 等入口。	React 页面可以挂载 Babylon 空场景。前置：M8-04。后续：M2-01。
M8-08 建立 CI	GitHub Actions/GitLab CI（按实际仓库）	每次提交执行 pnpm lint/build、dotnet test/build、C++ native test、WASM build、JSON Schema 校验。dev 合并必须全绿。	提交失败时能阻断合并。前置：M8-02~07。后续：全程。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M8-09 建立测试环境	Docker + Nginx + HTTPS	第 4 周前部署 test 环境；Web、API、PostgreSQL、assets 通过同一域名访问；日志滚动并保留。	外部浏览器可访问测试环境。前置：M4 基础/M1 壳。后续：M10。

M6 三维资产生产流水线

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M6-01 建立资产目录和命名	Blender + Git LFS/对象存储	为 B737-800 建立 assets/source/aircraft/b737-800/v001；原始下载文件只读保存；另建 working.blend，记录来源、许可、下载日期和 SHA256。	源文件、工作文件、发布文件不混用。前置：M0-01/02。后续：M6-02。
M6-02 Blender 导入审计	Blender	打开模型后检查单位、整体尺寸、面数、材质数、纹理、骨骼、动画、对象层级、起落架与机腹细节。截图记录“可用/需修/不可用”项。	产出 asset-audit.md；确认模型值得继续处理。前置：M6-01。后续：M6-03。
M6-03 清理无关结构	Blender	删除客舱、座椅、驾驶舱内部、小螺栓、不可见内件和训练无关高成本细节；保留机身外形、机腹、起落架、机翼、发动机外形和需要观察的底部区域。	近距离检查关键训练区域完整，面数明显下降。前置：M6-02。后续：M6-04。
M6-04 重建对象层级	Blender Collections	建立 AIRCRAFT_ROOT、RENDER、COLLIDER、ANCHOR、REGION、LOD0/1/2；起落架、机身、发动机等按需要拆分。对象名禁止 Plane.001 这类匿名名称。	Babylon 可以按稳定节点名定位对象。前置：M6-03。后续：M2-02。
M6-05 统一变换与原点	Blender	应用 Scale/Rotation；保证全机以米为单位；建立 AircraftRoot 原点与初始姿态；所有子模型挂到 Root 下，避免 Web 端逐个补偿旋转。	GLB 导入后尺寸与朝向一致。前置：M6-04。后续：M6-08。
M6-06 制作顶升点锚点	Blender Empty + metadata script	在经确认位置放置 Empty，命名 LP_*；每个锚点附加 ID/类型/状态，不把显示用小球当真正坐标源。	可脚本导出 lifting-points.json。前置：M0-02、M6-05。后续：M2-03、M3-04。
M6-07 制作安全/禁止区域	Blender 简化 Mesh / Polygon	把安全布设区和禁止区做成低面数不可见逻辑网格；显示版本另做半透明材质。区域节点使用 REGION_SAFE_* / REGION_FORBIDDEN_*。	区域既可视化又可导出判定数据。前置：M0-02。后续：M2-03、M3-03。
M6-08 制作碰撞代理	Blender	为机腹、起落架、地面和气囊交互区域制作简单 Box/Capsule/Convex collider；不要用全飞机高精网格作为碰撞体。	拾取/拖动/碰撞性能可控。前置：M6-04。后续：M2-04。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M6-09 制作 LOD0/1/2	Blender Decimate + 手工修复	LOD0 用于近看，LOD1 中景，LOD2 远景；每级检查轮廓、起落架和机腹关键形状不破坏。不要只看面数，必须在目标视距截图比较。	三套 LOD 可独立导出。前置：M6-03。后续：M6-12。
M6-10 整理材质与贴图	Blender/Substance	合并重复材质；普通零件降低纹理分辨率；保留必要法线/粗糙度；训练版采用中性涂装，避免不必要航空公司品牌资源。	无丢贴图、无粉色材质、材质槽数量受控。前置：M6-03。后续：M6-12。
M6-11 制作气囊/软管/阀门	Blender	按设备资料自制训练级模型；气囊建立 0/20/50/80/100% 形态或 Shape Key；软管只保留端头模型，管体由 Babylon 运行时生成。	设备模型都具有稳定 DeviceId 对应。前置：M0-06。后续：M2-04~06。
M6-12 导出 Web GLB	Blender glTF 2.0 Export	分别导出 aircraft_lod0/1/2.glb、devices.glb；检查材质、法线、尺度、节点名；发布包不得混入源 .blend。	Babylon Sandbox/本地测试可正确加载。前置：M6-05~11。后续：M6-13、M2-02。
M6-13 压缩与资产清单	KTX2/BasisU + glTF optimizer + Python	压缩纹理和几何；生成 asset-manifest.json，包含文件名、大小、hash、LOD、URL、兼容等级。压缩后逐个对比视觉差异。	Web Asset Pack 可按 manifest 加载。前置：M6-12。后续：M2-08。
M6-14 资产自动校验	Python	检查文件存在、hash、节点名、LOD、纹理尺寸、未授权品牌关键词、metadata schema；CI 中执行。	错误资产不能进入发布目录。前置：M6-13、M0-03。后续：M8-08。

M5 PostgreSQL 数据库

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M5-01 建立迁移工程	EF Core Migrations	API 中创建 DbContext；连接 dev PostgreSQL；所有 Schema 变更必须通过 Migration，不允许手工改生产库。	首次 migration 可从空库创建。前置：M8-03/05。后续：M5-02。
M5-02 建立目录表	PostgreSQL	创建 AircraftVersion、ScenarioVersion、RuleSetVersion、AssetPackage、DeviceCatalog；字段至少含 Id、Version、Status、CreatedAt、Metadata JSONB。	API 可以查询当前可用版本。前置：M0-03。后续：M4-03/M4-06。
M5-03 建立用户/角色	PostgreSQL + ASP.NET Identity 或简化自建	首期只保留 Admin/Instructor/Trainee；密码只存强哈希；账号状态、创建时间、最后登录时间可追踪。	支持登录和角色校验。前置：M5-01。后续：M4-02。
M5-04 建立 TrainingTask	PostgreSQL	保存任务创建人、机型版本、场景版本、规则版本、训练模式、状态、说明。任务创建时版本必须固定，不跟随后续更新。	一个任务可以创建训练会话。前置：M5-02/03。后续：M4-03。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M5-05 建立 TrainingSession	PostgreSQL	保存 SessionId、TaskId、Asset/Scenario/Rule Version、StartAt/EndAt、Status、客户端类型、最终结果。	会话创建后版本不可被静默替换。前置：M5-04。后续：M4-03/04。
M5-06 建立 TrainingEvent	PostgreSQL JSONB	字段：SessionId、Sequence、OccurredAt、Actor、ObjectId、Action、Parameters、StateBeforeHash、StateAfterHash、TriggeredRules、ScoreDelta。建立 SessionId+Sequence 唯一索引。	重复 Sequence 会被数据库/服务拒绝。前置：M3-05。后续：M4-04/M7。
M5-07 建立 Snapshot	PostgreSQL JSONB	只在关键步骤、规则触发和固定间隔保存；包含飞机、气囊、软管、阀门、StepState、核心版本号。	可以从任一快照继续事件回放。前置：M5-06。后续：M7-02/04。
M5-08 建立成绩表	PostgreSQL	ScoreSummary 保存总分和版本；ScoreItem 保存 RuleCode、事件 Sequence、扣分/加分、说明；ErrorRecord 保存错误严重程度和纠正状态。	每一分都能追溯到事件和规则。前置：M7-01。后续：M7-03。
M5-09 建立审计和幂等字段	PostgreSQL	关键写操作保存 RequestId/IdempotencyKey；管理员操作写 AuditLog；避免刷新或网络重试重复结束会话/重复扣分。	重复请求不会产生重复业务记录。前置：M5-04~08。后续：M4-04/05。
M5-10 备份恢复脚本	pg_dump/pg_restore + Docker	每日自动备份；测试环境至少执行一次从空库恢复；备份文件与运行日志保留。	有恢复记录而不是只有备份文件。前置：M8-09。后续：M10-05。

M4 ASP.NET Core 后端

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M4-01 建立 API 基础框架	ASP.NET Core	配置日志、异常中间件、ProblemDetails、CORS、OpenAPI、HealthCheck、环境变量、UTC 时间策略。所有 API 返回统一错误格式。	错误不会直接暴露堆栈给客户端。前置：M8-05。后续：全部 API。
M4-02 实现认证	ASP.NET Core Auth	实现 login/logout/me；首期使用 Cookie 或 JWT 二选一并固定；API 端基于角色授权，不靠前端隐藏按钮。	三种角色登录/拒绝用例通过。前置：M5-03。后续：M1-03。
M4-03 实现目录 API	EF Core + OpenAPI	GET aircraft/scenarios/devices/rules；只返回 Status=Approved 的正式数据，Draft 仅管理员可见。	任务页能拿到真实版本 ID。前置：M5-02。后续：M1-04。
M4-04 实现任务创建 API	POST /api/training/tasks	校验机型/场景/规则兼容性；写入 TrainingTask；把版本 ID 固化在任务上。	创建任务后数据不可因后续配置修改而变化。前置：M5-04。后续：M1-04。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M4-05 实现会话启动/结束	POST sessions/start, finish	StartSession 返回 SessionId、资源 manifest、场景配置、规则版本；FinishSession 进入 Finishing 状态，不直接相信客户端分数。	会话生命周期可追踪。前置：M5-05。后续：M2/M7。
M4-06 实现事件批量写入	POST /sessions/{id}/events:batch	每批按 Sequence 连续性校验；网络重试使用 IdempotencyKey；服务器只接受定义过的 Action。	乱序、重复、未知事件被明确拒绝。前置：M5-06/M3-05。后续：M7。
M4-07 实现快照写入/索引	API + PostgreSQL	客户端可提交经过 Schema 校验的 Snapshot；服务器在关键事件后可要求快照；记录 CoreVersion。	Replay API 能定位最近快照。前置：M5-07。后续：M7-04。
M4-08 接入 Native Simulation Core	C ABI + C# P/Invoke	构建 simulation-core native shared library；C# 封装 SimulationCoreNative；服务器结束会话时重新按事件回放，生成权威最终状态和规则触发结果。	正式考核分数不依赖客户端自报。前置：M3-01/05。后续：M7-03。
M4-09 实现成绩生成入口	POST /finish + Score Service	服务器重放事件→得到 RuleTriggered/StepCompleted→计算 ScoreSummary/ScoreItem→事务写库→Session=Completed。	同一事件集重复生成得到相同成绩。前置：M4-08/M7-01。后续：M1-05。
M4-10 实现回放查询	GET /replay	返回场景/规则/资产版本、最近 Snapshot、事件分页、错误定位、总时长，不返回每帧视频。	前端可按 Sequence 构建时间轴。前置：M5-06/07。后续：M7-04/M1-06。
M4-11 实现资产 Manifest API	GET /assets/packages/{id}	返回经过授权的静态资源 URL、hash、LOD、大小；二进制文件走 Nginx/对象存储，不塞 PostgreSQL。	浏览器按当前任务只下载一个机型包。前置：M6-13/M5-02。后续：M2-02。
M4-12 OpenAPI 客户端生成	openapi-typescript / codegen	CI 从后端 OpenAPI 生成 apps/web 的 typed client；业务页面禁止手写重复接口类型。	接口字段变更能在前端编译期暴露。前置：M4-01~11。后续：M1。
M4-13 日志与追踪	Serilog/OpenTelemetry（首期可只用结构化日志）	每条训练写日志时携带 SessionId/TaskId/UserId；异常日志不能包含密码、Token 或敏感配置。	出现问题可按 SessionId 追踪。前置：M4-01。后续：M10。
M4-14 API 异常与限流	ASP.NET Core	对事件批量接口设置合理大小限制；无效 JSON、超大 payload、未授权资源请求均返回明确错误。	异常输入不导致服务崩溃。前置：M4-06。后续：M10-05。

M3 Simulation Core (C++20 / WASM / Native)

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M3-01 定义纯核心边界	C++20	core 层不包含 DOM、Babylon、数据库、Unreal 头文件。输入是 Config/Command/Event，输出是 SimulationState/RuleResult/EventResult。	核心可在命令行单元测试中运行。前置：M8-06。后续：M3-02。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M3-02 定义 SessionState	C++ structs	建立 AircraftState、BagState、HoseState、ValveState、PressureState、StepState、RiskState；所有状态可序列化。	GetState() 可输出完整 JSON。前置：M0-03。后续：M3-03~05。
M3-03 实现训练状态机	C++	按 M0-04 流程配置实现 canExecute(command)、apply(command)、transition；禁止在 UI 点击时直接跳步。	正常流程和越序操作单测通过。前置：M0-04、M3-02。后续：M2-04~06。
M3-04 实现规则模型	C++ + rules.json	解析 RuleCode/AppliesTo/Condition/Severity/Action/Penalty/Version；先实现位置、连接、阀门、压力、同步偏差 5 类条件。	相同输入总是产生相同 RuleResult。前置：M0-05。后续：M7-01。
M3-05 定义命令与事件	C++/JSON Schema	Command：PlaceBag、RotateBag、ConnectHose、SetValve、SetPressure、ConfirmCheck、RecoverDevice；Event：命令成功/失败后的标准化结果。	Web、Server、Replay 共用同名 Action。前置：M0-03。后续：M2/M4/M7。
M3-06 气囊位置判定	几何算法	输入 Bag footprint/center/rotation 与 Allowed/Forbidden region；判断中心、包围盒/多边形是否在合法区域，并返回偏移量和原因。	边界、旋转、禁止区覆盖单测通过。前置：M6-07。后续：M2-04。
M3-07 管路拓扑	图结构	设备端口使用 PortId；ConnectHose 建立图边；校验端口类型、重复占用、禁止环路/错误连接（按业务规则）。	正确/错误端点均有明确结果。前置：M0-06。后续：M2-05。
M3-08 阀门与压力状态	状态机/配置曲线	ValveState 控制连通关系；PressureChanged 必须满足连接和步骤前置；压力上限/教学区间从 Approved 配置读取。	未连接先开阀、超阈值等能触发规则。前置：M3-07/M0-05。后续：M3-09。
M3-09 气囊充气曲线	配置驱动	每种气囊用 heightCurve/inflationLevels 定义 0/20/50/80/100% 的教学高度；不做真实软体力学。	相同充气级别输出稳定 supportHeight。前置：M0-06。后续：M3-10/M2-06。
M3-10 飞机姿态计算	简化刚体/平面拟合	根据有效支撑点高度与初始姿态计算教学用 Height/Pitch/Roll/SyncDeviation；算法输入输出必须可解释并可测试。	对称顶升、单侧偏高、前后偏高三个基准用例符合预期。前置：M3-09、M6-06。后续：M2-06。
M3-11 风险等级	规则 + 阈值配置	根据同步偏差、非法步骤、压力、支撑状态输出 Normal/Warning/Danger/Blocked；视觉颜色由 UI 决定，不写进核心。	风险等级可追溯到规则。前置：M3-04/08/10。后续：M2-07。
M3-12 快照序列化	JSON	实现 SerializeState/RestoreState；快照包含版本号和最小完整状态，不包含 Babylon 节点引用。	恢复后继续执行事件得到与原流程一致结果。前置：M3-02。后续：M7-02/04。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M3-13 WASM 适配	Emscripten	导出 createSession/applyCommand/getState/createSnapshot/restoreSnapshot/destroy; JS/TS wrapper 负责内存和 JSON 转换。	浏览器能加载并连续执行 1000 次命令无明显内存增长。前置：M3-01~12。后续：M2。
M3-14 Native C ABI	Shared Library	对同一核心增加稳定 C ABI，供 ASP.NET Core P/Invoke 和未来 Unreal adapter 使用；ABI 与核心内部类隔离。	服务端可重放一段固定事件并得到相同 state hash。前置：M3-12。后续：M4-08/M11-01。
M3-15 确定性测试	Unit tests	固定 Config + EventSet，Native 与 WASM 最终 StateHash、TriggeredRules、ScoreDelta 必须一致。	跨端一致性测试进入 CI。前置：M3-13/14。后续：M10-01。
M3-16 Mock 设备接口	C++ interfaces	定义 ITrackingDevice、IHapticDevice、IInputDevice；首期只有 Mock 实现，禁止 Web 代码直接引用任何厂商 SDK。	未来接硬件只新增 Adapter。前置：M3-01。后续：M11。

M2 Babylon.js Web 三维训练

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M2-01 建立 SceneController	Babylon.js	创建 Engine、Scene、Camera、Light、ground、resize、dispose；SceneController 对外只暴露 init/loadSession/dispose。	反复进入/退出训练页无重复 Canvas/监听器。前置：M8-07。后续：M2-02。
M2-02 建立 AssetLoader	Babylon SceneLoader	根据 AssetManifest 选择设备等级和 LOD；显示真实加载进度；校验 hash/404；所有资源挂到 SessionAssetRoot。	只下载当前任务资产；失败有重试/退出。前置：M4-11/M6-13。后续：M2-03。
M2-03 飞机根节点与相机	Babylon TransformNode/ArcRotateCamera	导入飞机后重新挂到 AircraftRoot；设置目标、最小/最大半径、上下视角限制；提供“全机/机腹/左主起落架/右主起落架”预设。	鼠标和触控操作稳定，不穿模到机身内部。前置：M6-05。后续：M2-04。
M2-04 点位图层	Babylon Mesh/GUI	根据 lifting-points.json 生成可开关标记；点击点位显示名称、适用设备、审核状态。Draft 点位只在开发模式显示。	显示坐标与 metadata 一致。前置：M6-06/M4-03。后续：M2-06。
M2-05 安全/禁止区可视化	Babylon translucent meshes	从 regions.json 生成半透明区域；正常状态低透明度，选择气囊时提高可见性；禁止区采用明显图案+文字，不只依赖颜色。	用户能一眼识别允许/禁止区域。前置：M6-07。后续：M2-08。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M2-06 设备栏	React + web-sim bridge	显示可用气囊、软管、阀门；选择设备后在 3D 创建 ghost preview；设备库存由 Scenario 配置，不在前端写死。	设备列表与当前任务一致。前置：M0-06/M4-03。后续：M2-07。
M2-07 气囊拖放	PointerDragBehavior / Ray Picking	拖动时只更新视觉 preview；落下时生成 PlaceBagCommand(position, rotation, deviceId) 发送 WASM；核心返回 Accepted/Rejected 和原因。	错误位置前端不能自己判定为正确。前置：M3-06。后续：M2-08。
M2-08 吸附/旋转/撤销	Babylon	核心返回推荐锚点/偏差后，前端可做视觉吸附；支持旋转、删除、撤销上一步；撤销同样走 Command/Event，不直接删除状态。	事件日志能完整记录撤销。前置：M2-07。后续：M2-09。
M2-09 管路端点交互	Picking + Tube/GreasedLine	点击端口 A 再点击端口 B 生成 ConnectHoseCommand；连接成功后以曲线/管线显示，失败只显示临时错误连线并提示原因。	正确/错误端点都有反馈。前置：M3-07、M6-11。后续：M2-10。
M2-10 阀门交互	Babylon mesh animation	阀门对象点击/旋转形成 SetValveCommand；UI 显示 Open/Closed；核心若阻断则视觉状态回退，不允许出现“画面开了但状态没开”。	视觉与 SimulationState 一致。前置：M3-08。后续：M2-11。
M2-11 压力控制面板	React HUD + Command	压力输入使用滑块/分级按钮；发送 SetPressureCommand；界面只显示核心返回的有效压力和风险，不直接修改气囊高度。	超限行为由规则决定并可回放。前置：M3-08。后续：M2-12。
M2-12 气囊充气动画	Morph Target/Shape Key + lerp	SimulationState.inflationRatio 驱动 0→1；若使用离散 Shape Key，在相邻状态间插值；动画层只做平滑，不改变最终数值。	刷新同一状态后外观可恢复。前置：M3-09/M6-11。后续：M2-13。
M2-13 飞机顶升动画	Quaternion/Transform interpolation	SimulationState 的 Height/Pitch/Roll 映射到 AircraftRoot；每帧插值追踪目标值；禁止直接改变子 Mesh。	无瞬移、抖动、累计漂移。前置：M3-10。后续：M2-14。
M2-14 训练 HUD	React Overlay	顶部：任务/步骤/连接状态；右侧：步骤说明/错误；底部：压力/阀门/继续；危险操作用文字+图标+颜色。HUD 不遮挡机腹关键区域。	桌面和目标平板布局均可操作。前置：M9-03。后续：M2-15。
M2-15 事件缓冲与提交	TS EventBuffer	每个 accepted command 生成标准 TrainingEvent；本地按 Sequence 缓存，每 N 条或 N 秒批量提交；断网保留未提交队列。	刷新前未提交事件有明确警告；重复提交后端幂等。前置：M4-06/M3-05。后续：M7。
M2-16 会话恢复	API + WASM restore	页面刷新/重新进入时获取最近 Snapshot + 后续 Events，在 WASM 中恢复状态，再一次性同步 Babylon 表现。	中断恢复不产生重复事件。前置：M4-10/M3-12。后续：M7-04。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M2-17 回放模式	ReplayController	训练模式禁用输入，按时间轴向 WASM 应用历史 Event；支持 pause/seek/speed；seek 时从最近 Snapshot 开始，不从头播完。	任意跳转后飞机/气囊/阀门状态正确。前置：M7-04。后续：M1-06。
M2-18 资源释放	Babylon dispose	退出训练页时 dispose scene、texture、mesh、observer、timer、event listener、WASM session；连续进入 20 次观察内存。	无持续内存上升。前置：M2-01~17。后续：M2-19。
M2-19 性能优化	Babylon Inspector + browser profiler	逐项优化三角面、纹理、Draw Call、透明物体、阴影、后处理、实例化、冻结静态材质；先资产后代码。	达到 performance-budget 或形成明确差距清单。前置：M6-13。后续：M10-04。
M2-20 降级策略	DeviceProfile	按设备检测选择 LOD/阴影/抗锯齿/纹理等级；低端模式优先保持交互和步骤提示，不追求最高画质。	同一 URL 可在 PC/平板自动选档。前置：M2-19。后续：上线。

M1 React Web 业务平台

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M1-01 应用壳与路由	React Router	建立 /login、/dashboard、/tasks/new、/training/:id、/results/:id、/replays、/replay/:id、/admin；加入登录守卫和 404。	所有页面可通过 URL 直接刷新。前置：M8-04。后续：M1-02。
M1-02 API Client	TanStack Query + generated client	封装 auth/catalog/task/session/score/replay；统一 loading/error/retry；服务端状态不重复塞进 Zustand。	接口失败有统一提示。前置：M4-12。后续：所有页面。
M1-03 登录与会话	React + ASP.NET Auth	登录页、退出、会话过期跳转、当前用户信息；前端角色只控制可见性，真实权限仍由 API 校验。	未登录不能打开训练 URL。前置：M4-02。后续：M1-04。
M1-04 Dashboard	React/Ant Design	展示开始训练、最近训练、成绩概览、待复训；首期不做复杂 BI。	可从首页进入任务创建。前置：M1-03。后续：M1-05。
M1-05 任务创建页	React Form	按机型→场景→训练模式联动；显示设备清单、资料版本、任务说明；提交后调用 create task + start session，再进入 Training。	不能选择不兼容版本。前置：M4-03/04/05。后续：M2。
M1-06 训练页容器	React + web-sim	页面负责 Session 数据加载、HUD、错误边界和 Babylon 容器；3D 逻辑全部在 web-sim 包。	三维包可独立测试，页面不直接操作 mesh。前置：M2-01。后续：训练。
M1-07 成绩页	React	显示总分、步骤得分、错误时间、RuleCode 的可读说明、训练时长、是否通过、进入回放按钮。	每个扣分项能定位到事件。前置：M4-09。后续：验收。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M1-08 训练记录页	React Table	按人员、时间、机型、场景、结果筛选；分页；点击进入成绩或回放。	数据多时不一次性全取。前置：M4-10。后续：M1-09。
M1-09 回放页	React + ReplayController	左侧事件/错误时间线，中间 3D，右侧状态与规则解释；点击错误可 seek 到对应 Sequence。	回放定位时间可接受。前置：M2-17/M7-04。后续：验收。
M1-10 Admin 只读目录	React	显示机型、场景、规则、资产版本及审核状态；首期不做复杂在线编辑器，避免配置写坏。	管理员可检查当前生产版本。前置：M4-03/11。后续：维护。
M1-11 全局异常/空状态	ErrorBoundary/Empty/Result	网络失败、资源失败、无数据、权限不足、会话已结束都有明确页面，不出现白屏。	故意断 API/资产请求可正确展示。前置：M1-01~10。后续：M10。
M1-12 响应式与触控适配	CSS/Ant Design breakpoints	优先支持横屏平板；按钮最小触控尺寸、侧栏折叠、HUD 缩放、禁止关键按钮过小。	指定终端可完成完整流程。前置：M9/M2。后续：M10-04。

M7 评分、事件与回放

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M7-01 定义评分构成	规则配置 + C++	总分拆为必做步骤、顺序、参数区间、严重错误、时间；所有 ScoreDelta 必须关联 RuleCode/Version 和 Event Sequence。	不存在“页面直接减 10 分”的代码。前置：M0-05/M3-04。后续：M7-03。
M7-02 定义一票否决	rules.json	明确哪些规则触发 Failed；一票否决仍要记录完整事件和失败原因；不把所有错误都做成阻断。	考核失败原因可解释。前置：M0-05。后续：M7-03。
M7-03 客户端即时分数	WASM	训练过程中可显示“暂估得分/错误”，但明确最终成绩由服务器 Native Core 重放生成。	客户端篡改分数不影响最终成绩。前置：M3-13/M4-08。后续：M1-07。
M7-04 服务器权威评分	Native Core + ASP.NET	Finish 时锁定事件序列→Native 重放→生成 RuleTriggered/StepCompleted→生成 ScoreSummary。	同一事件集多次计算结果一致。前置：M4-08/09。后续：M7-05。
M7-05 快照策略	Core + API	在 StepCompleted、严重 RuleTriggered、固定事件数量时创建 Snapshot；禁止每帧保存。	典型训练快照数量可控。前置：M3-12/M4-07。后续：M7-06。
M7-06 回放恢复算法	ReplayController + WASM	seek(targetSequence)：找 <= target 的最近 Snapshot→restore→依次 apply Event 到 target→同步 3D。	任意 seek 不需要从第 1 条事件开始。前置：M7-05。后续：M2-17。
M7-07 时间轴数据	API DTO	将 Event 按类别映射为可读标签：铺设、连接、阀门、压力、步骤、错误；严重错误在时间线上可筛选。	教员可快速定位错误。前置：M4-10。后续：M1-09。

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M7-08 回放一致性哈希	StateHash	关键 Sequence 保存 StateHash；回放到该点后重新计算并比对，发现不一致立即记录。	能够自动发现版本/算法破坏历史回放。前置：M3-12。后续：M10-03。
M7-09 版本冻结	DB + manifest	成绩和回放固定 Aircraft/Scenario/Rule/Asset/CoreVersion；旧会话不得自动升级资源。	旧记录在新版本上线后仍可解释。前置：M5-04。后续：维护。

M9 UI/UX 与视觉

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M9-01 建立 Design Tokens	Figma + packages/design-tokens	确定主色、灰阶、成功/警告/危险、字号、间距、圆角、阴影、边框、图标；输出 CSS variables/JSON。	React 与 3D HUD 使用同一 Token。前置：M0。后续：M1/M2。
M9-02 确定信息架构	Figma	首页→任务设置→训练→成绩→回放形成单主链；管理功能放次级入口，避免训练人员看到复杂后台。	5 次点击内能开始训练。前置：M9-01。后续：M9-03。
M9-03 设计训练页布局	Figma	中央 3D 最大化；顶部任务状态；左侧设备抽屉；右侧步骤/错误；底部压力与主操作。必须给机腹区域留出无遮挡视窗。	桌面与横屏平板均有高保真图。前置：M9-02。后续：M2-14。
M9-04 设计状态反馈	Figma	成功、警告、危险、阻断使用颜色+图标+文字；错误提示包含“发生了什么/为什么/如何纠正”，避免只有红框。	每条 RuleCode 有显示文案模板。前置：M0-05。后续：M2/M1。
M9-05 设计动效规则	CSS/Babylon	页面切换 150~250ms；3D 状态变化使用平滑插值；加载有真实进度；禁止无意义炫光和大面积动画。	动效服务于状态理解。前置：M9-01。后续：开发。
M9-06 空/错/加载状态	Figma	资产下载中、网络断开、无任务、无成绩、回放资源缺失、会话过期都单独设计。	开发不再用默认浏览器 alert。前置：M9-01。后续：M1-11。
M9-07 视觉一致性验收	UI QA checklist	逐页核对字号、间距、文案、图标、按钮层级、状态颜色；Babylon HUD 与 React 页面统一。	形成 UI QA 清单并清零高优先级问题。前置：M1/M2。后续：M10。

M10 测试、性能与验收

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M10-01 建立自动测试骨架	C++ tests + xUnit + Vitest	核心规则/状态机必须单测；API 服务至少覆盖任务、会话、事件、评分；前端关键工具函数测试。	CI 可自动运行。前置：M8-08。后续：全程。
M10-02 正常端到端用例	Playwright + 人工 3D 验收	创建任务→加载模型→铺设→连接→充气→顶升→稳压→泄压→回收→成绩→回放。	全过程无人工改数据库。前置：V0.3。后续：RC。
M10-03 错误用例集	验收矩阵	至少测试：错误区域、禁止区、错误气囊、错误端口、未连接开阀、超压、左右不同步、越序、重复操作、撤销、刷新、中断恢复。	每个错误都有 Rule/Event/Score/Replay 证据。前置：M0-07。后续：RC。
M10-04 回放一致性	自动脚本	随机抽取 Session，Native 重放和 Web WASM 回放比较 StateHash、最终姿态、规则、得分。	不一致必须阻断发布。前置：M7-08。后续：上线。
M10-05 性能基线	Chrome Performance/Babylon Inspector	记录首屏、资产下载、GPU/CPU 帧时间、内存、纹理、Draw Call、两小时稳定性；PC 与目标平板分别建报告。	达到预算或经确认调整预算。前置：M2-19/M6-13。后续：M10-06。
M10-06 性能问题处理顺序	资产→渲染→代码	先降纹理/面数/透明/阴影/Draw Call，再查逻辑；禁止第一反应推翻架构。每次优化前后保留数据。	性能改动有量化收益。前置：M10-05。后续：RC。
M10-07 浏览器/分辨率	Edge/Chrome + target tablet	测试 100%/125% 缩放、横屏、触控、刷新、全屏、网络慢速；明确正式支持范围。	支持矩阵写入交付文档。前置：M1-12。后续：RC。
M10-08 API 异常与安全	HTTP tests	未登录、越权、无效 ID、重复事件、乱序、超大 payload、过期 Session、资源无权访问。	服务不崩溃且错误可追踪。前置：M4。后续：RC。
M10-09 备份恢复	Docker/PostgreSQL	停止服务→恢复数据库→重新部署→登录→查看旧成绩和回放；验证版本资源仍能读取。	形成恢复演练记录。前置：M5-10/M8-09。后续：V1.0。
M10-10 最终验收	Acceptance Matrix	按用例逐项签字，不以截图、演示视频或“代码已经写完”代替功能验收。	所有 P0/P1 缺陷关闭。前置：RC1。后续：V1.0。

M11 未来 Unreal / VR / 力反馈预留

任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M11-01 固定 Native ABI	C API documentation	记录 CoreVersion、create/apply/getState/snapshot/restore 接口；禁止未来 Unreal 直接依赖 core 私有类。	可独立写 Unreal Adapter。前置：M3-14。

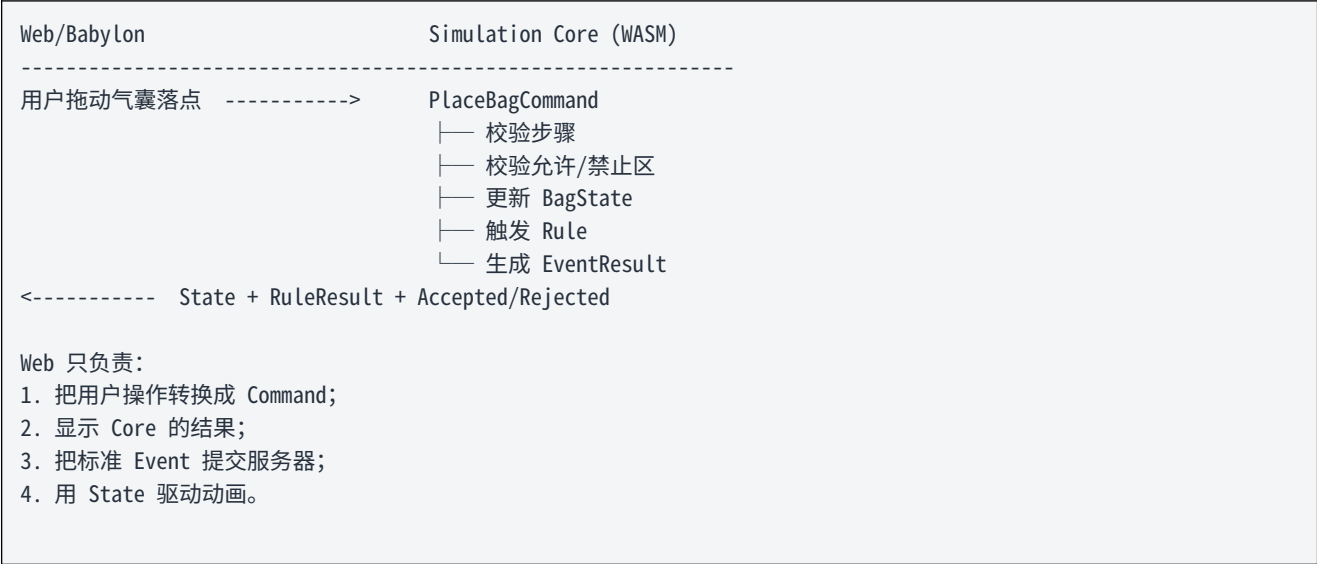
任务	用什么 / 文件	具体怎么做	输出、验收与依赖
M11-02 设备抽象契约	IHapticDevice/ITrackingDevice	定义位置、姿态、按钮、振动、力向量、设备健康状态、时间戳；首期只实现 Mock。	厂商 SDK 不污染训练业务。前置：M3-16。
M11-03 服务器接口稳定	OpenAPI	未来 Unreal 与 Web 使用同一 Login/Task/Session/Event/Finish/Replay API，不单独再做 VR 后台。	VR 只新增客户端。前置：M4。
M11-04 保留高质量资产	assets/source	Web 只发布轻量 GLB，但飞机/设备高质量 Source 永久保留；未来 Unreal 从 Source 重新导入，不反向放大 Web 模型。	资产可长期复用。前置：M6-01。
M11-05 事件模型扩展规则	contracts	未来灭火新增 ExtinguisherPicked、SafetyPinRemoved、TriggerPressed、AgentDischarged、FireSuppressed 等事件，但沿用 Session/Sequence/Rule/Score 结构。	新模块不复制训练平台。前置：M3-05。

5. 关键接口、数据契约与调用顺序

5.1 API 最小集合

方法	路径	用途
POST	/api/auth/login	登录；返回当前用户/会话
GET	/api/catalog/aircraft	可用具体机型版本
GET	/api/catalog/scenarios? aircraftVersionId=...	该机型可用场景
POST	/api/training/tasks	创建训练任务并固定版本
POST	/api/training/sessions	启动会话，返回资源 Manifest 和配置
POST	/api/training/sessions/{id}/ events:batch	批量写入标准事件
POST	/api/training/sessions/{id}/snapshot	写关键快照
POST	/api/training/sessions/{id}/finish	服务端重放并生成权威成绩
GET	/api/training/sessions/{id}/score	成绩和错误明细
GET	/api/training/sessions/{id}/replay	版本、快照、事件时间线
GET	/api/assets/packages/{id}	当前任务三维资源清单

5.2 浏览器与 Simulation Core 的固定边界



5.3 正式考核为什么要服务端重放

浏览器 WASM 负责即时反馈，但最终成绩不能直接信任客户端。训练结束后，ASP.NET Core 调用同一 Simulation Core 的 Native 版本，按数据库事件重新执行一次，生成权威成绩。这样既保留浏览器低延迟，也保证规则只有一份。

5.4 事件样例

```
{
  "sessionId": "ses_...",
  "sequence": 42,
  "occurredAt": "2026-09-01T10:15:32.120Z",
  "actor": "user_...",
  "objectId": "bag_02",
  "action": "BagPlaced",
  "parameters": {
    "deviceId": "AIRBAG_LOW_01",
    "position": [1.25, 0.0, -3.10],
    "rotationY": 15.0
  },
  "triggeredRules": [
    {
      "ruleCode": "BAG_OUTSIDE_ALLOWED_REGION",
      "version": "R1",
      "severity": "Warning",
      "penalty": -5
    }
  ],
  "scoreDelta": -5
}
```

6. 首期三维资产的具体处理标准

项目	执行标准
源文件	下载/采购原文件完整保存；建立许可台账；不直接在唯一源文件上删改
训练对象	机身外形、机腹、起落架、机翼、发动机外形必须保留；内部客舱等无关细节删除
对象命名	AircraftRoot / Fuselage / Gear_Main_L / Gear_Main_R / Engine_L / Engine_R 等稳定名称
逻辑层	COLLIDER、ANCHOR、REGION 与 RENDER 分开，训练判定不依赖高精网格
LOD	至少 LOD0/1/2，按实际目标设备确定面数，不以单一“越低越好”为目标
纹理	主体 2K 起步，普通零件 1K/512；压缩后必须人工比对关键标识与机腹细节
发布	GLB + 压缩纹理 + asset-manifest.json；按机型单独打包
未来 VR	高质量 Source 永久保留；Unreal 从 Source 重新导出，不直接拿 Web LOD0 当高质量模型

7. 16 周 / 80 工作日执行时间线

下面按工作日拆解。若某天前置任务未通过验收，不要用临时代码绕过，而应先解决前置问题。可以并行的任务可以同时推进，但版本门槛保持不变。

时间	当天必须完成的具体操作	涉及模块	当天产物/门槛
第 1 周：把所有工程“点亮”			
D1	冻结首期对象和场景；建立 scope.md、术语表、资料台账目录；创建 Git 仓库和目录结构。	M0-01/M0-03/M8-01	仓库 + Scope v1
D2	创建 React、ASP.NET Core、CMake 三个工程；配置格式化；Docker 启动 PostgreSQL。	M8-02~06	三端 Hello World
D3	定义数据库核心实体草图；创建首个 EF Migration；API /health；React 调 API 显示服务状态。	M5-01/M4-01/M1-02	Web→API→DB 贯通
D4	建立 Simulation Core 最小 SessionState；Native 单测；配置 Emscripten WASM 构建。	M3-01/M3-02/M8-06	Native/WASM 构建成功
D5	React 加载 WASM，调用 getState；CI 跑 Web/API/C++/WASM；形成 V0.0 技术链路截图和日志。	M8-08/M3-13	第一条技术竖切链
第 2 周：锁定模型与任务/会话			
D6	审核 B737-800 源模型的许可、面数、材质、机腹和起落架；复制为 working.blend。	M6-01/M6-02	资产审计
D7	清理无关结构、重建对象层级、统一尺度与 Root；导出第一版测试 GLB。	M6-03~05/M6-12	aircraft_test.glb
D8	Babylon 建 SceneController/AssetLoader/相机，加载 GLB，可旋转缩放。	M2-01~03	浏览器可看飞机
D9	数据库建立 Aircraft/Scenario/Rule/Asset/Device 版本表；API 输出 catalog。	M5-02/M4-03	目录 API
D10	任务创建与 StartSession；任务页选择机型/场景后进入训练页并显示 SessionId。	M4-04/05/M1-05/06	任务→会话→3D 贯通
第 3 周：顶升点与区域			
D11	在 Blender 建 LP_* 锚点和 REGION_*；写 metadata 导出/校验脚本。	M6-06/M6-07/M6-14	lifting-points/regions
D12	Babylon 加载点位、区域图层；实现图层开关和点位详情。	M2-04/M2-05	点位查看
D13	Simulation Core 状态机实现 Prepare/BagSelect/BagPlace；写正常/越序单测。	M3-03	最小训练状态机

时间	当天必须完成的具体操作	涉及模块	当天产物/门槛
D14	规则模型和首批 RuleCode；实现 Allowed/Forbidden region 位置判定。	M3-04/M3-06	位置规则
D15	会话结束/事件表 Schema 初版；训练页退出时正确 dispose；部署第一版 test 环境。	M5-05/06/M2-18/M8-09	V0.1 候选
第 4 周：V0.1 架构验收			
D16	实现标准 Command/Event Schema；前端提交一个测试事件，后端按 Sequence 入库。	M3-05/M4-06	Event pipeline
D17	建立 Figma Design Tokens 和训练页布局；同步到 React CSS variables。	M9-01~03	设计基线
D18	完善登录/主导航/任务页视觉；处理 loading/error/empty。	M1-01/M1-03/M1-11	基础 UI
D19	做 V0.1 全链路回归：登录→建任务→3D 飞机→点位→结束→数据库有会话/事件。	M10-01/M10-02	V0.1 测试记录
D20	修 V0.1 阻断问题；冻结技术栈、目录和 ID 规范。	全模块	V0.1 发布
第 5 周：气囊铺设			
D21	Blender 制作/整理气囊模型和 DeviceId；建立 0/20/50/80/100% 形态。	M6-11	气囊资产
D22	React 设备栏；Babylon ghost preview；从 Scenario 读取可用设备。	M2-06	设备选择
D23	实现气囊拖动、旋转、落点；生成 PlaceBagCommand。	M2-07	拖放
D24	WASM 判定位置并返回 Accepted/Rejected；前端根据结果吸附/回退/提示。	M3-06/M2-08	规则驱动铺设
D25	PlaceBag Event 入库；刷新后通过 Snapshot/Events 恢复铺设状态。	M2-15/M2-16/M4-06/07	可恢复铺设
第 6 周：管路闭环 V0.2			
D26	定义所有端口 PortId、设备连接约束；Core 实现 Hose graph。	M3-07	连接规则
D27	Babylon 端口拾取；连接成功用 Tube/Line 画管路，失败显示原因。	M2-09	管路交互
D28	实现撤销/重置命令，保证撤销也生成事件；测试重复连接/占用端口。	M2-08/M3-05	可纠错连接
D29	规则引擎支持 Warning/Blocked/Penalty/Failed；错误事件写库。	M3-04/M4-06	规则闭环

时间	当天必须完成的具体操作	涉及模块	当天产物/门槛
D30	V0.2 回归：正确/错误铺设+连接；UI 文案统一；发布 V0.2。	M9-04/M10-03	V0.2
第 7 周：阀门、压力、充气			
D31	Blender 完成阀门/压力设备模型；检查材质和动画 pivot。	M6-11	设备资产 v1
D32	Core 实现 ValveState、PressureState、连接前置条件、超限规则。	M3-08	阀门/压力逻辑
D33	训练页实现阀门点击/旋转与压力控制面板；全部发送 Command。	M2-10/M2-11	操作面板
D34	Core 建立气囊 inflationRatio/heightCurve；单测各级高度。	M3-09	充气状态
D35	Babylon Shape Key/Morph 动画由 inflationRatio 驱动；事件序列可重复播放。	M2-12	0→100% 气囊动画
第 8 周：完整顶升流程 V0.3			
D36	Core 实现支撑点、高度、Pitch/Roll/SyncDeviation；建立基准测试。	M3-10	姿态模型
D37	AircraftRoot 平滑顶升/俯仰/横滚；验证不累计漂移。	M2-13	飞机动画
D38	实现 Hold/Deflate/Recover 状态与规则；全流程能走到 Complete。	M3-03/M3-08	完整状态机
D39	补齐训练 HUD：步骤、压力、同步偏差、警告、下一步。	M2-14/M9-03	统一 HUD
D40	完成 V0.3 演示：铺设→连接→充气→顶升→稳压→泄压→回收；第 8 周后冻结首期大功能。	全模块	V0.3 核心闭环
第 9 周：评分系统			
D41	确定评分项和一票否决规则；所有规则绑定 RuleCode/Version。	M7-01/M7-02	评分表
D42	客户端即时 ScoreDelta/错误面板；明确“暂估”。	M7-03/M2-14	即时反馈
D43	构建 Native Core shared library；C# P/Invoke 调通固定事件重放。	M3-14/M4-08	服务器 Core
D44	FinishSession 服务端重放事件并生成 ScoreSummary/ScoreItem。	M4-09/M5-08/M7-04	权威成绩
D45	React 成绩页；每个扣分项跳到事件 Sequence；回归篡改客户端分数不影响最终值。	M1-07/M10-03	评分闭环
第 10 周：事件回放 V0.4			

时间	当天必须完成的具体操作	涉及模块	当天产物/门槛
D46	确定 Snapshot 触发点；实现 create/restore state。	M3-12/M7-05	快照
D47	Replay API 返回版本、快照索引、事件分页。	M4-10	回放数据
D48	ReplayController 实现从快照 seek 到 Sequence。	M7-06/M2-17	可 seek 回放
D49	回放 UI：时间轴、错误筛选、暂停/倍速、点击错误定位。	M1-09/M7-07	回放页
D50	StateHash 一致性检查；发布 V0.4。	M7-08/M10-04	V0.4
第 11 周：界面完成度			
D51	按 Figma 收口 Dashboard/任务/训练/成绩/回放；清理开发态临时按钮。	M9/M1	高保真 UI
D52	统一组件尺寸、字重、状态色、对话框、通知、表格、空状态。	M9-07	UI QA v1
D53	优化训练 HUD 遮挡、触控尺寸、横屏平板布局。	M1-12/M2-14	平板布局
D54	补齐网络断开、资源失败、会话过期、权限不足流程。	M1-11/M2-02	异常体验
D55	邀请非开发人员按脚本完成一次训练，记录所有“看不懂/找不到/误点”问题并修复。	M9/M10	可用性问题清单
第 12 周：性能 V0.5			
D56	统计模型三角面、纹理、材质、Draw Call、资产包大小；确定最大成本项。	M6/M2-19	性能画像
D57	优先处理纹理和模型：LOD、KTX2、合材质、删除不可见对象。	M6-09/10/13	资产优化 v2
D58	处理渲染：阴影、透明、后处理、实例化、冻结静态材质。	M2-19	渲染优化
D59	PC 与目标平板测试首载/FPS/内存/触控；启用 DeviceProfile 降级。	M2-20/M10-05	性能报告
D60	若性能未过预算，停止加功能继续优化；过线后发布 V0.5。	M10-06	V0.5
第 13 周：异常与数据正确性			
D61	执行 10+ 类错误操作；核对 Rule/Event/Score/Replay 四者一致。	M10-03	错误测试
D62	测试重复提交、乱序事件、页面刷新、重复 finish；修幂等。	M5-09/M4-06	幂等报告
D63	测试断网后事件缓冲与恢复；确认事件 Sequence 连续。	M2-15	网络恢复

时间	当天必须完成的具体操作	涉及模块	当天产物/门槛
D64	测试 Draft/Approved 数据权限和版本冻结；旧 Session 不跟随新规则。	M5-02/M7-09	版本测试
D65	核心规则、状态机、跨端确定性单测全部进入 CI；P0 缺陷清零第一轮。	M3-15/M10-01	稳定候选
第 14 周：RC1			
D66	运行 2 小时稳定性；连续进入/退出训练页；检查内存增长。	M2-18/M10-05	稳定性报告
D67	浏览器/平板/分辨率兼容矩阵；确定正式支持边界。	M10-07	兼容矩阵
D68	数据库备份、清库、恢复；验证旧成绩/回放。	M5-10/M10-09	恢复记录
D69	API 越权、无效输入、超大 payload、资源权限测试。	M10-08	安全/异常报告
D70	冻结 RC1；从此只修 Bug，不新增功能。	全模块	RC1
第 15 周：生产化与文档			
D71	建立 production compose、Nginx、HTTPS、环境变量、日志目录、健康检查。	M8-09/M12	生产部署脚本
D72	首次生产环境部署；运行 smoke test；确认静态资产缓存头。	M8/M10	生产候选
D73	整理 API 文档、数据库 ER/表说明、资产规范、版本策略。	docs	开发文档
D74	整理用户手册：任务、训练、错误、成绩、回放；管理员只写首期实际功能。	docs	用户文档
D75	整理第三方组件、模型许可、购买凭证、修改记录、版本 hash。	M6/M0	资产台账
第 16 周：验收与 V1.0			
D76	按 acceptance-matrix 完整跑正常流程和全部错误流程。	M10-10	验收记录 v1
D77	随机抽 3 个会话核对数据库事件、服务器重放、Web 回放、最终成绩。	M10-04	一致性终检
D78	修最后 P0/P1；重跑对应回归；冻结版本号、CoreVersion、AssetVersion。	全模块	Release Candidate final
D79	准备演示数据、演示账号、演示脚本；清理测试脏数据和调试开关。	M1/M4/M5	交付环境
D80	发布 V1.0；归档源代码、数据库 Migration、资产 Source/Web、测试报告和部署手册；建立 Phase 2 backlog。	全模块	V1.0 正式交付

8. 关键依赖关系与不能颠倒的顺序

依赖	原因
M0 → M6	不先确定具体子型号/训练区域就开始大量减面，会返工。
M6 → M2	模型层级、坐标、锚点、碰撞代理不稳定时，不要写大量 Babylon 交互。
M0 → M3	状态机和规则只实现已经定义的流程；未知阈值必须配置化而不是程序员猜。
M3 ↔ M2	M2 只采集输入和表现，M3 才决定操作是否允许、姿态和风险。
M3 → M4/M7	事件、规则和 StateHash 先统一，数据库和回放才能稳定。
M5 → M4	Schema/Version/Index 先设计，API 再围绕业务写；禁止在控制器里拼 SQL。
M9 → M1/M2	设计系统第 1~4 周就落地，避免最后一周“补美化”。
M7 → M1 成绩/回放	先保证事件和评分可解释，再做漂亮页面。
M2/M6 → M10 性能	性能问题优先检查资产和渲染，不先推翻整套技术栈。
M3 Native → 未来 Unreal	未来 VR 复用 Core/事件/API，不重新实现气囊或灭火评分平台。

9. 每个里程碑的“Definition of Done”

版本	必须具备	门槛
V0.1 / 第 4 周	登录→创建任务→启动 Session→加载飞机→查看点位/区域→结束 Session；数据库有版本化会话和事件。	不允许再大改技术栈/目录。
V0.2 / 第 6 周	气囊可拖放；正确/错误位置由 Core 判定；管路可连接；规则与事件入库。	三维表现与规则彻底分离。
V0.3 / 第 8 周	从铺设到回收完整走完；气囊和飞机动画由 Core state 驱动。	首期大功能冻结。
V0.4 / 第 10 周	服务器权威评分、事件、快照、回放、错误定位完成。	系统达到“培训系统”而非三维展示。
V0.5 / 第 12 周	高保真 UI；目标设备性能达到预算或经过确认调整；DeviceProfile 可降级。	停止新增功能。
RC1 / 第 14 周	功能、错误用例、兼容、稳定、备份恢复、权限基本通过。	只修 Bug。
V1.0 / 第 16 周	部署、文档、资产台账、验收记录、源代码、数据库迁移全部归档。	正式交付。

10. 首期数据库对象与字段最小集

对象	最小字段
AircraftVersion	Id, Family, Subtype, Version, Status, MetadataJson, CreatedAt
ScenarioVersion	Id, AircraftVersionId, FaultType, Version, InitialStateJson, RuleSetVersionId, Status
RuleSetVersion	Id, Version, RulesJson, Status, ApprovedAt
AssetPackage	Id, AircraftVersionId, Version, ManifestUrl, Hash, Status
DeviceCatalog	Id, DeviceType, Version, SpecJson, AssetPackageId, Status
TrainingTask	Id, CreatedBy, AircraftVersionId, ScenarioVersionId, RuleSetVersionId, Mode, Status
TrainingSession	Id, TaskId, AssetVersion, ScenarioVersion, RuleVersion, CoreVersion, StartAt, EndAt, Status
TrainingEvent	Id, SessionId, Sequence, OccurredAt, Action, ObjectId, ParametersJson, TriggeredRulesJson, ScoreDelta
Snapshot	Id, SessionId, Sequence, StateJson, StateHash, CoreVersion, CreatedAt
ScoreSummary	Id, SessionId, TotalScore, Passed, RuleVersion, CoreVersion, GeneratedAt
ScoreItem	Id, ScoreSummaryId, Sequence, RuleCode, Delta, Description
AuditLog	Id, UserId, Action, TargetType, TargetId, PayloadJson, CreatedAt

11. 当前首期与未来 VR/力反馈之间如何衔接

首期 Web 气囊模块完成后，不是把 Babylon 页面“改成 VR”，而是新增 Unreal 客户端。可以复用的部分与需要新增的部分如下：

类别	实现方式
直接复用	账号/任务/会话 API、PostgreSQL 数据、事件模型、规则版本、评分、回放思想、Simulation Core Native、源资产
新增	Unreal 场景、OpenXR 交互、VR UI、火焰/烟雾/灭火剂视觉、设备 Adapter
力反馈	在 C++ Adapter 中把统一 IHapticDevice 映射到具体厂商 SDK；业务模块只发目标力/振动，不直接调用设备 SDK
实体灭火器	追踪器、扳机、压力/位置传感器作为 Device Input；高频设备控制留在本地原生客户端，不经过浏览器
新培训模块	新增 SimulationModule + Scene + RuleSet + AssetPackage；不复制登录/成绩/数据库/回放系统

12. 开发中最容易出现的 12 个错误

- 1. 把顶升点、阈值、评分写死在 React/Babylon 代码里，而不是版本化配置和 Simulation Core。
- 2. 下载一个飞机模型后立即开始交互开发，没有先固定节点层级、坐标、碰撞代理和资产许可。
- 3. 用高精飞机网格直接做碰撞，导致浏览器拾取和内存成本异常。
- 4. 为了做软管而第一版就上绳索物理，浪费时间且回放不稳定。
- 5. 气囊动画自己决定“高度”，Core 又有一套高度，最后视觉与业务状态分裂。
- 6. 客户端直接提交最终分数，正式考核容易被篡改且不同端结果不一致。
- 7. 回放录屏而不是事件+快照，无法解释扣分和状态。
- 8. 先做八架飞机再验证一架的完整流程，最终把架构问题复制八遍。
- 9. 把 Web 减面模型当唯一资产，未来 Unreal 没有高质量 Source 可用。
- 10. 第 15 周才第一次上服务器，部署、CORS、HTTPS、缓存、路径问题一起爆发。
- 11. 只用颜色表示危险/正确，平板或视觉条件下信息不清晰。
- 12. 性能问题一出现就换框架，而没有先测纹理、面数、Draw Call、透明、阴影和内存。

13. 开工前 10 项检查清单

状态	检查项
☐	具体子型号和故障已经冻结
☐	飞机模型许可与源文件已归档
☐	顶升点/安全区/禁止区资料已有 Draft 或 Approved 版本
☐	Git 仓库和目录已创建
☐	React/API/PostgreSQL/C++ Native/WASM 全部能本地启动
☐	Web 能调用 API，Web 能调用 WASM
☐	第一版 GLB 能在 Babylon 加载且尺寸正确
☐	状态机流程名称与 UI 术语统一
☐	事件/版本/ID 命名已经固定
☐	第 4 周 V0.1 的验收脚本已经写好

14. 最终交付物清单

类别	交付内容
代码	React Web、Babylon web-sim、ASP.NET Core API、C++ Simulation Core Native/WASM、Python tools
数据库	PostgreSQL Schema、Migration、种子数据、备份/恢复脚本
三维资产	高质量 Source、Web GLB/LOD、设备模型、碰撞代理、点位/区域、Manifest、许可台账

类别	交付内容
功能	任务、训练、铺设、连接、阀门/压力、顶升、稳压/泄压/回收、评分、事件、快照、回放
测试	单元、端到端、错误用例、回放一致性、性能、兼容、稳定性、恢复
部署	Docker Compose、Nginx、HTTPS、环境变量模板、日志和健康检查
文档	需求基线、接口、数据库、资产规范、部署、用户手册、验收矩阵
扩展预留	Native ABI、Device Contract、Source 资产、事件扩展规范

15. 执行结论

这套方案的重点不是“把技术都用上”，而是让首期每一天都有可验收产物，同时把未来 VR、灭火和力反馈需要的边界一次切对。四个月内只完成一架飞机并不算少；只要完整流程稳定、界面成熟、事件/评分可解释、回放可复现、性能达标，并且未来新机型和 Unreal 客户端能在现有接口上继续扩展，这个首期就是成功的。

第一天真正要做的事情

不要先写复杂页面，也不要先做八架飞机。先冻结 B737-800（或资料最完整的具体子型号）和一个故障场景；同时把 Git、React、API、PostgreSQL、C++ Native/WASM 五条链路点亮。第 5 个工作日必须看到 Web 能调 API，也能调 WASM。